

BOOK 09 — JDBC & DATABASE

Beginner to Production Mastery (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 09 of 24 **Java Versions**

Covered: JDBC 4.x (try-with-resources support since Java 7), core SQL concepts version-agnostic

Prerequisites: Book 01–02 (classes, interfaces), Book 04 (exception handling — SQLException), Book 05 (Collections — building result lists), Book 08 (connection pooling touches concurrency)

Next Book: 10_Advanced_Java_Backend.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ఇప్పటివరకు మన data అంతా in-memory (Collections) లో ఉంది — application restart అయితే పోతుంది. ఈ పుస్తకం లో Java application ని **relational database** తో connect చేసి, persist చేయడం ఎలాగో నేర్చుకుంటాము — `java.sql` /JDBC API నుండి మొదలుపెట్టి, transactions, connection pooling, SQL fundamentals వరకు. ఇది Book 13 (JPA/Hibernate) కి direct foundation.

English: All our data so far has lived in-memory (Collections) — lost on application restart. This book teaches connecting a Java application to a relational database and persisting data — starting from the `java.sql` /JDBC API, through transactions and connection pooling, to SQL fundamentals. This is the direct foundation for Book 13 (JPA/Hibernate).

Learning Objectives

1. Explain JDBC architecture and the driver model.
2. Connect to a database and manage `Connection` lifecycle correctly.
3. Master `Statement`, `PreparedStatement` (and why it matters for security), and `CallableStatement`.
4. Process `ResultSet`s correctly and safely.
5. Understand transactions, ACID properties, and commit/rollback.
6. Understand transaction isolation levels and the concurrency problems they solve.
7. Understand and use connection pooling.
8. Refresh core SQL: joins and normalization.
9. Understand indexes and basic query optimization.
10. Apply production-grade JDBC patterns (DAO pattern, resource management).

Table of Contents

Ch.	Title
1	JDBC Architecture Overview

Ch.	Title
2	Connecting to a Database
3	Statement, PreparedStatement & CallableStatement
4	ResultSet Deep Dive
5	Transactions, ACID, Commit & Rollback
6	Transaction Isolation Levels & Locking
7	Connection Pooling
8	SQL Fundamentals Refresher: Joins & Normalization
9	Indexes & Query Optimization
10	Production JDBC Patterns + Mini Project
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — JDBC Architecture Overview

Telugu Explanation

JDBC (Java Database Connectivity) అనేది Java application లు relational databases తో communicate చేయడానికి ఇచ్చే **standard API** — database-agnostic గా. Actual database-specific logic **JDBC Driver** (ఒక్కో database vendor తన driver ఇస్తుంది — MySQL Connector/J, PostgreSQL JDBC Driver) లో ఉంటుంది. మీ Java code ఎప్పుడూ `java.sql` interfaces మీదనే program చేస్తుంది, ఏ underlying database అయినా — driver మారితే code మారదు.

Professional English Explanation

JDBC (Java Database Connectivity) is the standard, database-agnostic API Java applications use to communicate with relational databases. The actual database-specific communication logic lives in a **JDBC Driver** (each vendor provides one — MySQL Connector/J, PostgreSQL JDBC Driver, etc.). Your Java code always programs against the `java.sql` interfaces, never a vendor-specific class directly — swapping the underlying database (in principle) doesn't require changing your code, only the driver/connection URL.

Diagram — JDBC Architecture

```
Your Java Application
|
v
java.sql API (Connection, Statement, ResultSet - INTERFACES, database-agnostic)
|
v
JDBC Driver Manager ---selects the right driver based on the connection URL---
|
v
Vendor-specific JDBC Driver (MySQL / PostgreSQL / Oracle / H2 ...)
|
v
Actual Database Server (network protocol specific to that vendor)
```

Java Code — Minimal End-to-End Example

```
import java.sql.*;

public class JdbcArchitectureDemo {
    public static void main(String[] args) {
        String url = "jdbc:h2:mem:testdb";           // H2 in-memory DB - great for
        learning/testing, no install needed
        String user = "sa";
        String password = "";

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            System.out.println("Connected! Driver: " + conn.getMetaData().getDriverName());
            System.out.println("Database product: " +
            conn.getMetaData().getDatabaseProductName());

            try (Statement stmt = conn.createStatement()) {
                stmt.execute("CREATE TABLE employee (id INT PRIMARY KEY, name VARCHAR(100),
                salary DOUBLE)");
                stmt.execute("INSERT INTO employee VALUES (1, 'Ravi', 70000.0)");

                try (ResultSet rs = stmt.executeQuery("SELECT * FROM employee")) {
                    while (rs.next()) {
                        System.out.println(rs.getInt("id") + " - " + rs.getString("name") + " -
                        " + rs.getDouble("salary"));
                    }
                }
            } catch (SQLException e) {
                System.out.println("Database error: " + e.getMessage());
            }
        }
    }
}
```

Output

```
Connected! Driver: H2 JDBC Driver
Database product: H2
1 - Ravi - 70000.0
```

Internal Working

- `DriverManager.getConnection(url, ...)` inspects the URL's prefix (`jdbc:h2:`, `jdbc:mysql:`, `jdbc:postgresql:`) to find a registered driver claiming to handle it — since JDBC 4.0 (Java 6+), drivers self-register automatically via the `java.sql.Driver` service-provider mechanism (no more manual `Class.forName("...")` calls needed, which was required in older JDBC/Java versions).
- `Connection`, `Statement`, `ResultSet` are all `AutoCloseable` (Book 04, Ch.5) — this is exactly why try-with-resources is the idiomatic, safe way to use them; forgetting to close a `Connection` leaks a real, finite database resource (a connection slot on the DB server), which is a serious, common production bug.
- The nested try-with-resources structure (`Connection` → `Statement` → `ResultSet`) mirrors the natural resource-ownership hierarchy — a `ResultSet` is meaningless without its `Statement`,

which is meaningless without its `Connection`, and closing an outer resource is documented to also close resources it opened, though explicit nested closing remains the clearer, safer idiom.

Real-World Example

Telugu: Real backend applications JDBC ని direct గా వాడటం అరుదు — దాదాపు ఎప్పుడూ Spring Data JPA/Hibernate (Book 13) వాడతారు, ఇది internal గా JDBC `me`ీదనే build అయ్యింది. ఈ book లో JDBC నేర్చుకోవడం, ఆ higher-level abstractions ఎలా పనిచేస్తాయో అర్థం చేసుకోవడానికి foundational. English: Real backend applications rarely use raw JDBC directly — almost everyone uses Spring Data JPA/Hibernate (Book 13), which is itself built entirely on top of JDBC internally. Learning JDBC here is foundational precisely because it demystifies what those higher-level abstractions are actually doing underneath.

Interview Answer

"JDBC is Java's standard, database-agnostic API for relational database access. Application code programs against `java.sql` interfaces (`Connection`, `Statement`, `ResultSet`); vendor-specific `Driver` implementations handle the actual wire protocol. Since JDBC 4.0, drivers self-register automatically. `Connection` / `Statement` / `ResultSet` are all `AutoCloseable`, so `try-with-resources` is the idiomatic way to guarantee they're released."

Cross Questions

- Q: Why don't you need `Class.forName("com.mysql.cj.jdbc.Driver")` anymore in modern JDBC?
→ A: Since JDBC 4.0 (Java 6+), drivers implement the service-provider mechanism and self-register when their JAR is on the classpath, discovered automatically by `DriverManager`.
- Q: Is JDBC code truly 100% portable across different databases? → A: Mostly, at the JDBC API level — but actual SQL syntax differences (vendor-specific functions, pagination syntax, etc.) mean real portability requires care, which is one of the practical reasons ORMs like Hibernate (Book 13) exist — to abstract over those SQL dialect differences too.
- Q: What happens if you forget to close a `Connection`? → A: It leaks — the database server continues holding that connection slot open, and under load this can exhaust the database's max-connections limit, a real and serious production incident category.

Coding Exercise

L1: Set up an H2 in-memory database connection, create a table, insert a row, and query it back. **L2:** Print the `DatabaseMetaData` (driver name, product version, JDBC major/minor version) for your connection. **L3:** Deliberately "forget" to close a `Connection` (don't use `try-with-resources`) and discuss why this is risky, even though the demo program still worked. **L4 (Interview):** Draw the JDBC architecture diagram from memory. **L5 (Senior):** Explain why direct JDBC usage is rare in modern applications, and what problem Book 13's JPA/Hibernate solves on top of it. **L6 (Mastery):** Explain, from memory, how `DriverManager` selects the correct driver for a given connection URL.

CHAPTER 2 — Connecting to a Database

Telugu Explanation

Connection URL format: `jdbc:<subprotocol>://<host>:<port>/<database>` — ఒక్కో database కి syntax slight గా వేరుగా ఉంటుంది. Connection object ఒక **expensive** resource (network handshake, authentication) కాబట్టి, దీన్ని efficiently manage చేయడం ముఖ్యం — direct గా create చేయడం కంటే connection pooling (Ch.7) production లో వాడతారు.

Professional English Explanation

Connection URL format: `jdbc:<subprotocol>://<host>:<port>/<database>` — syntax varies slightly per database vendor. A Connection object is an **expensive** resource to establish (network handshake, authentication), which is why managing it efficiently matters — production code uses connection pooling (Ch.7) rather than creating raw connections directly for every operation.

Java Code

```
import java.sql.*;
import java.util.Properties;

public class ConnectionDemo {
    public static void main(String[] args) throws SQLException {
        // Different real-world connection URL formats (illustrative, not run - no live servers here)
        String mysqlUrl = "jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC";
        String postgresUrl = "jdbc:postgresql://localhost:5432/mydb";
        String h2InMemoryUrl = "jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1"; // actually
        // runnable, no server needed

        System.out.println("MySQL URL format: " + mysqlUrl);
        System.out.println("PostgreSQL URL format: " + postgresUrl);

        // Connecting with Properties (alternative to passing user/password directly)
        Properties props = new Properties();
        props.setProperty("user", "sa");
        props.setProperty("password", "");

        try (Connection conn = DriverManager.getConnection(h2InMemoryUrl, props)) {
            System.out.println("Auto-commit is ON by default: " + conn.getAutoCommit());
            // true by default
            System.out.println("Connection is valid: " + conn.isValid(2));
            // timeout in seconds
            System.out.println("Read-only: " + conn.isReadOnly());

            conn.setAutoCommit(false);
            // manual transaction control (Ch.5)
            System.out.println("Auto-commit now: " + conn.getAutoCommit());
            conn.setAutoCommit(true);
            // reset for demo simplicity
        }
        System.out.println("Connection automatically closed by try-with-resources.");
    }
}
```

Output

```
MySQL URL format: jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC
PostgreSQL URL format: jdbc:postgresql://localhost:5432/mydb
Auto-commit is ON by default: true
Connection is valid: true
Read-only: false
Auto-commit now: false
Connection automatically closed by try-with-resources.
```

Internal Working

- Establishing a `Connection` involves a real TCP handshake, authentication round-trip, and often SSL/TLS negotiation with the database server — this is genuinely expensive (often measured in tens of milliseconds), which is precisely why creating a fresh connection per request/query in a high-throughput application is a serious performance anti-pattern, motivating connection pooling (Ch.7).
- Auto-commit mode** (`true` by default) means every individual SQL statement is automatically wrapped in and committed as its own implicit transaction — convenient for simple one-off statements, but incorrect for any operation needing multiple statements to succeed or fail together atomically (Ch.5's ACID), which requires explicitly calling `setAutoCommit(false)`.
- `Connection.isValid(timeout)` performs a lightweight check (often a trivial query or a driver-specific ping) to confirm the connection is still usable — connections can silently die (network issues, DB server restart, idle timeout) without the JVM immediately knowing, which is exactly the kind of problem connection pools (Ch.7) actively guard against via validation.

Real-World Example

Telugu: Production applications `application.properties / application.yml` (Book 12) లో connection URL, credentials externalize చేస్తాయి — hardcode చేయరు, security మరియు environment-specific configuration (dev/staging/prod) కోసం. English: Production applications externalize the connection URL and credentials into configuration files (Book 12's `application.properties / application.yml`) rather than hardcoding them — both for security (credentials shouldn't be in source code/version control) and for environment-specific configuration across dev/staging/production.

Interview Answer

"A JDBC connection URL follows `jdbc:<subprotocol>://<host>:<port>/<database>`, vendor-specific in exact syntax. Establishing a `Connection` is expensive (network handshake, auth), which is why production code pools connections rather than creating them per-operation. Auto-commit is on by default, wrapping each statement in its own implicit transaction — multi-statement atomic operations require explicitly disabling it via `setAutoCommit(false)`."

Cross Questions

- Q: Why is auto-commit dangerous for multi-step operations? → A: Each statement commits independently and immediately — if a later statement in a logically-related sequence fails,

earlier ones are already permanently committed, leaving the database in an inconsistent intermediate state instead of atomically rolling back everything.

- Q: Why shouldn't credentials be hardcoded in source code? → A: Security — hardcoded credentials in version control are a well-known, serious real-world breach vector; externalized configuration (environment variables, secrets managers, config files excluded from version control) is standard practice.
- Q: What does `Connection.isValid()` actually check? → A: Whether the underlying connection is still genuinely usable (not silently dropped by the network/DB server), typically via a lightweight round-trip, within the given timeout.

Tricky Questions

- Q: If auto-commit is `true` and a `SELECT` statement fails midway (e.g., network error), is there anything to "roll back"? → A: Not meaningfully for a read-only `SELECT` — auto-commit's atomicity concern is specifically about statements that *modify* data (`INSERT` / `UPDATE` / `DELETE`); reads don't have transactional state to lose.
- Q: Does closing a `Connection` automatically roll back any uncommitted changes if auto-commit was disabled? → A: Behavior here is driver/database-dependent and risky to rely on — the safe, correct practice is to always explicitly `commit()` or `rollback()` before closing a connection with auto-commit disabled, never leaving it to chance (Ch.5 covers this fully).

Coding Exercise

L1: Connect to an H2 in-memory database and print its auto-commit status, validity, and read-only flag. **L2:** Toggle `setAutoCommit(false)`, perform 2 inserts, and (without committing yet) discuss what state the data is in. **L3:** Write connection URLs for 3 different database vendors (MySQL, PostgreSQL, Oracle) based on documentation research. **L4 (Interview):** Explain why raw connection creation per request is a performance anti-pattern. **L5 (Senior):** Design a configuration strategy for externalizing database credentials across dev/staging/production environments. **L6 (Mastery):** Explain, from memory, why auto-commit mode is unsafe for multi-statement atomic operations.

CHAPTER 3 — Statement, PreparedStatement & CallableStatement

Telugu Explanation

`Statement` static SQL execute చేయడానికి (parameters లేకుండా). `PreparedStatement` **parameterized** SQL కి — precompiled, `?` placeholders తో, **SQL Injection నుండి రక్షిస్తుంది** మరియు repeated execution కి faster. `CallableStatement` stored procedures call చేయడానికి.

Professional English Explanation

`Statement` executes static SQL (no parameters). `PreparedStatement` handles **parameterized** SQL — precompiled with `?` placeholders — critically **protecting against SQL Injection** and offering better performance for repeated execution. `CallableStatement` invokes stored procedures.

Java Code

```
import java.sql.*;

public class StatementTypesDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb2", "sa", "")) {
            try (Statement setup = conn.createStatement()) {
                setup.execute("CREATE TABLE users (id INT PRIMARY KEY, username VARCHAR(50), password VARCHAR(50))");
                setup.execute("INSERT INTO users VALUES (1, 'admin', 'secret123')");
            }

            // DANGEROUS: Statement with string concatenation - SQL INJECTION VULNERABLE
            String userInput = "admin' OR '1'='1"; // classic injection
            payload

            String unsafeSql = "SELECT * FROM users WHERE username = '" + userInput + "'";
            System.out.println("Unsafe query (DO NOT DO THIS): " + unsafeSql);
            try (Statement stmt = conn.createStatement()) {
                ResultSet rs = stmt.executeQuery(unsafeSql) {
                    int count = 0;
                    while (rs.next()) count++;
                    System.out.println("Unsafe query returned " + count + " row(s) - injection bypassed the intended filter!");
                }

            // SAFE: PreparedStatement - the '?' placeholder is NEVER interpreted as SQL syntax
            String safeSql = "SELECT * FROM users WHERE username = ?";
            try (PreparedStatement pstmt = conn.prepareStatement(safeSql)) {
                pstmt.setString(1, userInput); // same malicious input, safely parameterized
                try (ResultSet rs = pstmt.executeQuery()) {
                    int count = 0;
                    while (rs.next()) count++;
                    System.out.println("Safe PreparedStatement query returned " + count + " row(s) - injection neutralized.");
                }
            }

            // PreparedStatement for INSERT - typed parameter binding
            String insertSql = "INSERT INTO users (id, username, password) VALUES (?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(insertSql)) {
                pstmt.setInt(1, 2);
                pstmt.setString(2, "ravi");
                pstmt.setString(3, "ravipass");
                int rowsAffected = pstmt.executeUpdate();
                System.out.println("Rows inserted: " + rowsAffected);
            }

            // Batch execution for multiple similar statements - much faster than individual round-trips
            try (PreparedStatement batchStmt = conn.prepareStatement(insertSql)) {
                for (int i = 3; i <= 5; i++) {
                    batchStmt.setInt(1, i);
                    batchStmt.setString(2, "user" + i);
                    batchStmt.setString(3, "pass" + i);
                    batchStmt.addBatch();
                }
                int[] results = batchStmt.executeBatch();
                System.out.println("Batch insert count: " + results.length);
            }
        }
    }
}
```

```
}  
    }  
}  
}
```

Output

```
Unsafe query (DO NOT DO THIS): SELECT * FROM users WHERE username = 'admin' OR '1'='1'  
Unsafe query returned 1 row(s) - injection bypassed the intended filter!  
Safe PreparedStatement query returned 0 row(s) - injection neutralized.  
Rows inserted: 1  
Batch insert count: 3
```

Internal Working

- `Statement.executeQuery(sql)` sends the SQL string to the database **exactly as constructed** — if that string was built via naive concatenation of untrusted user input, the database has no way to distinguish "data" from "SQL syntax," which is the entire mechanism behind SQL Injection: the attacker's input (`' OR '1'='1'`) is interpreted as SQL logic, not a literal string value.
- `PreparedStatement` sends the SQL **template** (with `?` placeholders) to the database **separately** from the parameter values — the database precompiles the query structure once, and parameter values are bound afterward purely as **data**, never re-parsed as SQL syntax. This is a structural, not merely a "sanitization," defense — it's not that special characters are escaped, it's that the data channel and the code channel are fundamentally separated.
- **Batch execution** (`addBatch()` / `executeBatch()`) sends multiple parameterized statements to the database in one network round-trip (or a small number thereof, depending on the driver/JDBC batch size settings), dramatically reducing the overhead of many individual `executeUpdate()` calls — a genuine, measurable production performance technique for bulk inserts/updates.

Real-World Example

Telugu: SQL Injection ఇప్పటికీ OWASP Top 10 లో ఉంది — real production breaches ఇప్పటికీ string-concatenated SQL వల్ల జరుగుతున్నాయి. `PreparedStatement` వాడటం ఒక optional best practice కాదు, **mandatory security requirement** ఏ user input involve అయిన query కైనా. English: SQL Injection remains on the OWASP Top 10, and real production breaches still happen from string-concatenated SQL — using `PreparedStatement` for any query involving user input isn't an optional best practice, it's a mandatory security requirement, and this chapter's demo showing the actual bypass is exactly why that requirement is non-negotiable.

Interview Answer

"`Statement` executes static SQL with no parameters — unsafe for any query involving user input, since string concatenation opens the door to SQL Injection. `PreparedStatement` sends the SQL structure and parameter values separately, so the database never re-interprets parameter data as SQL syntax — a structural defense against injection, not just character escaping — plus better performance for repeated execution via precompilation and batching. `CallableStatement` invokes stored procedures."

Cross Questions

- Q: Why is `PreparedStatement` a structural defense against SQL Injection, not just "escaping special characters"? → A: Because the SQL template is sent and compiled by the database separately from the parameter values — the parameters are bound as pure data afterward, never re-parsed as part of the SQL grammar at all, regardless of what characters they contain.
- Q: What performance benefit does `PreparedStatement` offer beyond security? → A: The database can cache/reuse the compiled query plan across repeated executions with different parameter values, and batch execution (`addBatch()`) reduces network round-trips for bulk operations.
- Q: When would you use `CallableStatement` instead of `PreparedStatement`? → A: When invoking a database stored procedure (pre-written SQL logic stored and executed on the database server itself), especially one with `OUT` or `INOUT` parameters that `PreparedStatement` can't handle.

Tricky Questions

- Q: Does `PreparedStatement` protect against SQL Injection if you still concatenate untrusted input directly into the SQL template string (not via `?` / `setX()`)? → A: No — the protection specifically comes from using `?` placeholders and `setX()` binding for the untrusted data; concatenating raw input into the template string itself (e.g., a dynamic table/column name) reintroduces the exact same vulnerability, since that part of the string still gets parsed as SQL syntax.
- Q: Can `PreparedStatement` parameters be used for table or column names? → A: No — `?` placeholders can only bind to *values* (data), never to SQL structural elements like table/column names or `ORDER BY` direction; those require careful allowlist-based validation if they must be dynamic, since they can't go through the safe parameter-binding mechanism at all.

Coding Exercise

L1: Reproduce the SQL Injection demo (unsafe `Statement` bypass) and the safe `PreparedStatement` fix, side by side. **L2:** Write a `PreparedStatement`-based INSERT and UPDATE for a custom table. **L3:** Use `addBatch()` / `executeBatch()` to insert 100 rows and compare (conceptually or by timing) against 100 individual `executeUpdate()` calls. **L4 (Interview):** Explain precisely why `PreparedStatement` prevents SQL Injection at a structural level. **L5 (Senior):** Review a codebase using string-concatenated `Statement` queries for user-facing search functionality — identify the vulnerability and provide the `PreparedStatement` fix. **L6 (Mastery):** Explain, from memory, why `?` placeholders can bind to values but never to table/column names, and what to do when a dynamic identifier is genuinely needed.

CHAPTER 4 — ResultSet Deep Dive

Telugu Explanation

`ResultSet` query results ని represent చేస్తుంది — ఒక **cursor** లాగా, ఒక్కసారి ఒక్క row మీద position అవుతుంది. `next()` cursor ని తర్వాతి row కి move చేస్తుంది, `boolean` return చేస్తుంది (ఇంకా rows ఉన్నాయా అని). Column access **index-based** (1-indexed, 0 కాదు!) లేదా **name-based** గా చేయచ్చు.

Professional English Explanation

`ResultSet` represents query results as a **cursor** — positioned on one row at a time. `next()` moves the cursor to the next row, returning a `boolean` indicating whether a row exists. Column access can be **index-based** (1-indexed, not 0!) or **name-based**.

Java Code

```
import java.sql.*;
import java.util.*;

public class ResultSetDemo {
    record Employee(int id, String name, double salary) {}

    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb3", "sa", "")) {
            try (Statement setup = conn.createStatement()) {
                setup.execute("CREATE TABLE emp (id INT, name VARCHAR(50), salary DOUBLE)");
                setup.execute("INSERT INTO emp VALUES (1, 'Ravi', 70000), (2, 'Anitha', 85000), (3, 'Kiran', 60000)");
            }

            List<Employee> employees = new ArrayList<>();
            try (PreparedStatement pstmt = conn.prepareStatement("SELECT id, name, salary FROM emp ORDER BY id");
                ResultSet rs = pstmt.executeQuery()) {

                ResultSetMetaData meta = rs.getMetaData();
                System.out.println("Column count: " + meta.getColumnCount());
                for (int i = 1; i <= meta.getColumnCount(); i++) {
                    System.out.println("  Column " + i + ": " + meta.getColumnName(i) + " (" + meta.getColumnTypeName(i) + ")");
                }

                while (rs.next()) { // cursor advances;
                    // false when no more rows
                    int id = rs.getInt(1); // index-based (1-indexed!)
                    String name = rs.getString("name"); // name-based (both valid)
                    double salary = rs.getDouble("salary");
                    employees.add(new Employee(id, name, salary));

                    // getInt()/getDouble() etc. on a NULL column returns 0/0.0 - must check wasNull() to distinguish!
                    if (rs.wasNull()) System.out.println("  (last read column was actually SQL NULL)");
                }

                System.out.println("Mapped to domain objects: " + employees);

                // Scrollable ResultSet (must request explicitly - forward-only is the default)
                try (Statement scrollStmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_READ_ONLY);
                    ResultSet rs = scrollStmt.executeQuery("SELECT * FROM emp ORDER BY id")) {
                    rs.last(); // jump to the last row

                    System.out.println("Last row id: " + rs.getInt("id"));
                    rs.first(); // jump back to the first row

                    System.out.println("First row id: " + rs.getInt("id"));
                    System.out.println("Total row count: " + rs.getRow() + " onward - use rs.last()+rs.getRow() to count");
                }
            }
        }
    }
}
```

```
}  
}
```

Output

```
Column count: 3  
Column 1: ID (INTEGER)  
Column 2: NAME (CHARACTER VARYING)  
Column 3: SALARY (DOUBLE PRECISION)  
Mapped to domain objects: [Employee[id=1, name=Ravi, salary=70000.0], Employee[id=2,  
name=Anitha, salary=85000.0], Employee[id=3, name=Kiran, salary=60000.0]]  
Last row id: 3  
First row id: 1  
Total row count: 1 onward - use rs.last()+rs.getRow() to count
```

Internal Working

- A `ResultSet` is **not** a fully-materialized in-memory collection by default — many drivers stream rows from the database as `next()` is called (especially important for very large result sets, avoiding loading millions of rows into memory at once); this is exactly why a `ResultSet` can only safely be consumed while its parent `Statement / Connection` remains open, and why mapping to domain objects (like the `Employee` record above) inside the loop, before closing, is the standard idiom.
- Column indices are **1-based**, not 0-based — a frequent source of off-by-one bugs for developers coming from array/list conventions (Book 01, Ch.5; Book 05) elsewhere in Java.
- `getInt()` / `getDouble()` / other primitive-returning getters return `0 / 0.0 / false` for a SQL `NULL` value (since primitives can't represent `null`, Book 01 Ch.12) — `wasNull()` must be called immediately after to distinguish "the value was genuinely 0" from "the value was NULL," a classic, easy-to-miss correctness bug; using the wrapper-type getters (`getObject()`), or a nullable-aware mapping layer) avoids this ambiguity entirely.
- By default, a `ResultSet` is **forward-only** (`TYPE_FORWARD_ONLY`) — you can only call `next()` , never go backward or jump around; scrollable result sets (`TYPE_SCROLL_INSENSITIVE / TYPE_SCROLL_SENSITIVE`) must be explicitly requested at `Statement` creation and carry additional driver/database overhead, so they're used only when genuinely needed (e.g., certain pagination implementations).

Real-World Example

Telugu: DAO methods `ResultSet` ని domain objects (`Employee` , `Order`) గా map చేసి, `ResultSet / Connection` close అయిన తర్వాత కూడా వాడగలిగే `List<Employee>` return చేస్తాయి — ఇది Book 13లో JPA/Hibernate automatic గా చేసేది, ఇక్కడ మీరు manual గా చేస్తున్నారు. English: DAO methods map each `ResultSet` row into a domain object (`Employee` , `Order`) and return a `List` that remains usable after the `ResultSet / Connection` are closed — this is exactly the "object-relational mapping" that Book 13's JPA/Hibernate automates for you, and understanding this manual version here is what makes that automation meaningful rather than magical.

Interview Answer

"`ResultSet` is a cursor over query results, positioned one row at a time via `next()`. Column access can be by 1-based index or by column name. Primitive getters return 0/false for SQL NULL, requiring `wasNull()` to disambiguate. By default it's forward-only and often streams rows rather than materializing them all in memory, which is why it must be consumed — typically mapped into domain objects — before its parent `Connection/Statement` closes."

Cross Questions

- Q: Why is checking `wasNull()` necessary after calling a primitive-returning getter? → A: Because `getInt()` / `getDouble()` /etc. can't represent SQL NULL (Java primitives have no null value, Book 01 Ch.12), so they silently return 0/0.0 for both "the value is actually 0" and "the value is NULL" — `wasNull()` is the only way to tell them apart.
- Q: Can you call `rs.getString("name")` after the underlying `Connection` has been closed? → A: No — a `ResultSet` is invalidated once its parent `Statement / Connection` closes; you must fully process and map its data to your own objects before closing anything.
- Q: What's the default `ResultSet` type, and what does it restrict? → A: `TYPE_FORWARD_ONLY` — you can only move forward via `next()`, never backward or to an arbitrary row, unless you explicitly request a scrollable type at `Statement` creation.

Tricky Questions

- Q: If a table has a nullable `INT` column and a row's value is genuinely NULL, what does `rs.getInt("that_column")` return, and how do you correctly detect the NULL? → A: It returns 0 (not an error, not `null`) — you must immediately call `rs.wasNull()` afterward, which returns `true` in this case, to correctly distinguish it from an actual stored value of 0.
- Q: Is `ResultSetMetaData` available even if the query returns zero rows? → A: Yes — `getMetaData()` describes the *shape* of the result (column names, types, count) independent of how many rows were actually returned, so it's available immediately after `executeQuery()` regardless of row count.

Coding Exercise

L1: Query a table and map every row into a custom domain record, printing the resulting list. **L2:** Deliberately query a nullable column with a NULL value using `getInt()`, and demonstrate the `wasNull()` check. **L3:** Use a scrollable `ResultSet` to jump to the last row, then the first row, then iterate forward from the beginning again. **L4 (Interview):** Explain why column indices in `ResultSet` are 1-based, and why that's a common source of bugs. **L5 (Senior):** Design a DAO method that safely maps a `ResultSet` to a `List<T>`, correctly handling nullable columns, before the connection closes. **L6 (Mastery):** Explain, from memory, why a `ResultSet` typically can't be consumed after its parent `Connection` is closed.

CHAPTER 5 — Transactions, ACID, Commit & Rollback

Telugu Explanation

Transaction అంటే ఒక్క **logical unit of work** గా treat చేయబడే multiple database operations — అన్నీ succeed అవ్వాలి (`commit`), లేదా అన్నీ fail అవ్వాలి (`rollback`), మధ్యస్థ state ఉండకూడదు. **ACID** ఈ guarantee ని formalize చేస్తుంది: **Atomicity** (all-or-nothing), **Consistency** (valid state నుండి valid state కి మాత్రమే), **Isolation** (concurrent transactions ఒకదానికొకటి interfere కాకూడదు, Ch.6), **Durability** (commit అయిన తర్వాత permanent గా ఉండాలి, even crash తర్వాత కూడా).

Professional English Explanation

A transaction is a group of database operations treated as one logical unit of work — either all succeed (`commit`) or all fail (`rollback`), with no partial/intermediate state ever persisted. **ACID** formalizes this guarantee: **Atomicity** (all-or-nothing), **Consistency** (only valid-state-to-valid-state transitions), **Isolation** (concurrent transactions don't interfere with each other, Ch.6), and **Durability** (once committed, changes survive even a subsequent crash).

Java Code — The Classic Bank Transfer Example

```
import java.sql.*;

public class TransactionDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb4", "sa", "")) {
            try (Statement setup = conn.createStatement()) {
                setup.execute("CREATE TABLE account (id INT PRIMARY KEY, balance DOUBLE)");
                setup.execute("INSERT INTO account VALUES (1, 1000.0), (2, 500.0)");
            }

            System.out.println("Before transfer: " + printBalances(conn));

            // SUCCESSFUL transfer - both operations commit together
            transferMoney(conn, 1, 2, 200.0);
            System.out.println("After successful transfer: " + printBalances(conn));

            // FAILING transfer - simulate a mid-transaction error, verify ROLLBACK leaves
            // state unchanged
            try {
                transferMoneyWithSimulatedFailure(conn, 2, 1, 100.0);
            } catch (SQLException e) {
                System.out.println("Transfer failed and was rolled back: " + e.getMessage());
            }
            System.out.println("After failed transfer (unchanged, rolled back): " +
                printBalances(conn));
        }
    }

    static void transferMoney(Connection conn, int fromId, int toId, double amount) throws
    SQLException {
        conn.setAutoCommit(false); // start manual
        // transaction
        try {
            try (PreparedStatement debit = conn.prepareStatement("UPDATE account SET balance =
            balance - ? WHERE id = ?")) {
                debit.setDouble(1, amount);
                debit.setInt(2, fromId);
                debit.executeUpdate();
            }
            try (PreparedStatement credit = conn.prepareStatement("UPDATE account SET balance =
            balance + ? WHERE id = ?")) {
                credit.setDouble(1, amount);
                credit.setInt(2, toId);
                credit.executeUpdate();
            }
            conn.commit(); // BOTH
        } catch (SQLException e) {
            conn.rollback(); // EITHER
            throw e;
        } finally {
            conn.setAutoCommit(true);
        }
    }

    static void transferMoneyWithSimulatedFailure(Connection conn, int fromId, int toId, double
    amount) throws SQLException {

```

```

        conn.setAutoCommit(false);
        try {
            try (PreparedStatement debit = conn.prepareStatement("UPDATE account SET balance =
balance - ? WHERE id = ?")) {
                debit.setDouble(1, amount);
                debit.setInt(2, fromId);
                debit.executeUpdate(); // this
WILL succeed and be visible in this transaction...
            }
            throw new SQLException("Simulated failure BEFORE the credit step"); // ...but
never gets committed
        } catch (SQLException e) {
            conn.rollback(); // undoes
the debit too - true atomicity
            throw e;
        } finally {
            conn.setAutoCommit(true);
        }
    }

    static String printBalances(Connection conn) throws SQLException {
        StringBuilder sb = new StringBuilder();
        try (Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT id, balance FROM account ORDER BY id")) {
            while (rs.next())
                sb.append("acct").append(rs.getInt("id")).append("=").append(rs.getDouble("balance")).append("
");
        }
        return sb.toString();
    }
}

```

Output

```

Before transfer: acct1=1000.0 acct2=500.0
After successful transfer: acct1=800.0 acct2=700.0
Transfer failed and was rolled back: Simulated failure BEFORE the credit step
After failed transfer (unchanged, rolled back): acct1=800.0 acct2=700.0

```

Internal Working & the 4 ACID Properties

Property	Guarantee	What Breaks Without It
Atomicity	All operations in a transaction succeed, or none do	A crash mid-transfer could debit one account without crediting the other — money vanishes
Consistency	Only valid states are ever persisted (constraints, triggers honored)	A transaction could leave a foreign key pointing to a deleted row
Isolation	Concurrent transactions don't see each other's uncommitted changes (levels vary, Ch.6)	A concurrent read could see a "half-transferred" balance
Durability	Once <code>commit()</code> returns, the change survives even a subsequent crash	A server crash right after "successful" payment confirmation could silently lose the transaction

- `conn.rollback()` undoes **everything** done since the transaction began (since the last `commit()` or the connection's start) — this is why `transferMoneyWithSimulatedFailure` correctly leaves the debit un-persisted even though the `executeUpdate()` for it had already "succeeded" from the JDBC call's perspective — the database only makes it permanent at `commit()`.
- The `try { ... } catch (SQLException e) { rollback(); throw e; } finally { setAutoCommit(true); }` shape in this chapter's demo is the standard, essential idiom for manual transaction management — always pair a transaction's `commit()` with a `rollback()` in the failure path, and always restore `autoCommit` state afterward.

Real-World Example

Telugu: Bank transfer, e-commerce order placement (inventory decrement + order creation + payment charge — అన్నీ ఒకే transaction లో), ఏదైనా ఒక్కటి fail అయితే మిగతావి కూడా rollback అవ్వాలి — ఇది ACID యొక్క అత్యంత classic, real-world justification. English: Bank transfers and e-commerce order placement (inventory decrement + order creation + payment charge, all needing to succeed or fail together) are the textbook real-world justification for ACID transactions — this chapter's transfer example is literally the industry-standard teaching example because it's exactly the kind of operation where partial failure is unacceptable.

Interview Answer

"A transaction groups multiple database operations into one atomic unit — `commit()` persists all of them together, `rollback()` undoes all of them together, with no partial state ever surviving. ACID formalizes the guarantee: Atomicity (all-or-nothing), Consistency (valid states only), Isolation (concurrent transactions don't interfere, levels covered in Ch.6), and Durability (committed data survives crashes). In JDBC, this requires disabling auto-commit and explicitly calling `commit()` / `rollback()`, always pairing them via try/catch."

Cross Questions

- Q: What does `rollback()` actually undo? → A: Every operation performed since the transaction began (since the connection was opened with auto-commit off, or since the last `commit/rollback`) — restoring the database to its state before the transaction started.
- Q: Why must auto-commit be disabled for multi-statement atomic operations? → A: With auto-commit on, each statement commits immediately and independently — there's no way to undo an earlier statement if a later one in the same logical operation fails.
- Q: What specifically does Durability guarantee, and why does it matter? → A: Once `commit()` successfully returns, the change is guaranteed to survive even a subsequent crash (typically via write-ahead logging on the database server) — without it, a "successful" transaction could still be silently lost.

Tricky Questions

- Q: If `debit.executeUpdate()` succeeds (returns without throwing) but the transaction is later rolled back, was the debit "really" applied? → A: Only within that in-progress, uncommitted transaction's own view — other connections/transactions wouldn't see it (per isolation, Ch.6),

and once rolled back, it's as if it never happened at all; JDBC's `executeUpdate()` succeeding only means the statement itself executed without error, not that its effects are permanent.

- Q: Is it safe to call `commit()` twice in a row without any statements in between? → A: It's generally a harmless no-op (there's nothing new to commit), but relying on this isn't good practice — the code should track transaction state clearly rather than calling `commit()` / `rollback()` speculatively.

Coding Exercise

L1: Implement the bank transfer example yourself, verifying both the success and simulated-failure paths. **L2:** Deliberately omit the `rollback()` call in the failure path and observe the resulting inconsistent state (careful — only in a throwaway test database). **L3:** Extend the transfer method to validate sufficient balance before debiting, throwing a custom exception (Book 04, Ch.6) that triggers rollback. **L4 (Interview):** Explain all 4 ACID properties with a one-sentence "what breaks without it" for each. **L5 (Senior):** Design an e-commerce order-placement transaction spanning inventory decrement, order creation, and payment charge — identify what should trigger rollback and why. **L6 (Mastery):** Explain, from memory, the standard try/catch/finally shape for manual JDBC transaction management, and why each part is necessary.

CHAPTER 6 — Transaction Isolation Levels & Locking

Telugu Explanation

Multiple transactions concurrently run అయినప్పుడు, మూడు classic problems రావచ్చు: **Dirty Read** (ఇంకా commit కాని data చదవడం), **Non-Repeatable Read** (ఒకే transaction లో, ఒకే row ని రెండుసార్లు చదివితే వేర్వేరు values రావడం, మధ్యలో వేరే transaction commit చేసినందుకు), **Phantom Read** (ఒకే query రెండుసార్లు run చేస్తే, వేర్వేరు row **count** రావడం). SQL Standard 4 isolation levels define చేస్తుంది, ఈ problems ని ఎంత avoid చేస్తారో బట్టి: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, **SERIALIZABLE**.

Professional English Explanation

When multiple transactions run concurrently, three classic problems can occur: **Dirty Read** (reading another transaction's uncommitted data), **Non-Repeatable Read** (re-reading the same row within one transaction yields a different value, because another transaction committed a change in between), and **Phantom Read** (re-running the same query within one transaction yields a different *row count*). The SQL standard defines four isolation levels, trading consistency guarantees against concurrency performance: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, **SERIALIZABLE**.

Isolation Levels Table

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Concurrency
READ_UNCOMMITTED	❌ Possible	❌ Possible	❌ Possible	Highest
READ_COMMITTED	✅ Prevented	❌ Possible	❌ Possible	High (most databases' default)
REPEATABLE_READ	✅ Prevented	✅ Prevented	❌ Possible (in some DBs)	Medium
SERIALIZABLE	✅ Prevented	✅ Prevented	✅ Prevented	Lowest

Java Code

```
import java.sql.*;

public class IsolationLevelsDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb5", "sa", "")) {
            System.out.println("Default isolation level (driver-reported constant): " +
                conn.getTransactionIsolation());
            System.out.println("TRANSACTION_READ_COMMITTED = " +
                Connection.TRANSACTION_READ_COMMITTED);
            System.out.println("TRANSACTION_SERIALIZABLE = " +
                Connection.TRANSACTION_SERIALIZABLE);

            conn.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);           //
            strictest - explicit choice
            System.out.println("Set to SERIALIZABLE: " + conn.getTransactionIsolation());

            conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);         //
            typical production default
            System.out.println("Set to READ_COMMITTED: " + conn.getTransactionIsolation());

            // Pessimistic locking preview (real multi-connection demo requires 2 separate
            connections + threads,
            // conceptually shown here via the SQL that would be used):
            System.out.println("""

                Pessimistic locking example (SELECT ... FOR UPDATE):
                Connection A: SELECT balance FROM account WHERE id=1 FOR UPDATE;  --
            locks the row
                Connection B: UPDATE account SET balance=... WHERE id=1;          --
            BLOCKS until A commits/rolls back
                This prevents a classic 'lost update' at the DATABASE level, complementing
                (not replacing) the in-JVM concurrency control from Book 08.
                """);
        }
    }
}
```

Output

```
Default isolation level (driver-reported constant): 2
TRANSACTION_READ_COMMITTED = 2
TRANSACTION_SERIALIZABLE = 8
Set to SERIALIZABLE: 8
Set to READ_COMMITTED: 2

Pessimistic locking example (SELECT ... FOR UPDATE):
  Connection A: SELECT balance FROM account WHERE id=1 FOR UPDATE;  -- locks the row
  Connection B: UPDATE account SET balance=... WHERE id=1;          -- BLOCKS until A
commits/rolls back
This prevents a classic 'lost update' at the DATABASE level, complementing
(not replacing) the in-JVM concurrency control from Book 08.
```

Internal Working

- Databases implement isolation primarily via either **locking** (a transaction acquires locks on rows/ranges it reads/writes, blocking conflicting concurrent access — used by `SERIALIZABLE` in many databases) or **MVCC (Multi-Version Concurrency Control)** (each transaction sees a consistent snapshot of the data as of when it started or as of each statement, without blocking readers — how PostgreSQL/MySQL InnoDB implement `READ_COMMITTED` / `REPEATABLE_READ` efficiently) — the exact mechanism is database-specific, though the *observable* isolation guarantees follow the SQL standard's levels.
- Higher isolation levels provide stronger consistency guarantees but reduce concurrency (more blocking, more lock contention, or more overhead maintaining snapshots) — this is a direct, deliberate trade-off, exactly analogous to Book 08's throughput-vs-latency GC trade-offs, and Book 08's lock-granularity trade-offs — choosing the *lowest* isolation level that's still safe for your specific use case is the general guidance, not blindly defaulting to `SERIALIZABLE`.
- **`SELECT ... FOR UPDATE`** (pessimistic locking) explicitly locks the selected rows for the duration of the transaction, forcing other transactions attempting to modify (or, depending on the database, even read) those same rows to wait — this is the database-level analog to Book 08's `synchronized` /locks, and is commonly used for the exact same reason: preventing a lost-update race condition, but at the database layer, protecting against concurrent access from *multiple application instances/connections*, not just multiple threads within one JVM.

Real-World Example

Telugu: E-commerce checkout లో, రెండు concurrent requests ఒకే product యొక్క చివరి stock unit కొనాలని try చేస్తే — `SELECT ... FOR UPDATE` (లేదా optimistic locking, version column తో) వాడకపోతే, overselling జరిగే ప్రమాదం ఉంది. ఇది Book 05, Ch.3's race condition కి, ఇప్పుడు database-level equivalent. English: Two concurrent checkout requests both trying to buy a product's last unit of stock is the classic scenario where `SELECT ... FOR UPDATE` (or optimistic locking via a version column) prevents overselling — the direct database-level analog to Book 08's in-JVM race conditions, now happening across potentially many separate application server instances all sharing one database.

Interview Answer

"Concurrent transactions can suffer dirty reads, non-repeatable reads, and phantom reads, in decreasing order of severity. The SQL standard's four isolation levels — `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE` — progressively prevent more of these at the cost of reduced concurrency. Databases implement isolation via locking or MVCC. Pessimistic locking (`SELECT ... FOR UPDATE`) explicitly locks rows for a transaction's duration, the database-level analog to Book 08's in-JVM locks, protecting against races across multiple application instances/connections."

Cross Questions

- Q: What is a dirty read, and which isolation level first prevents it? → A: Reading another transaction's uncommitted (possibly-to-be-rolled-back) data — `READ_COMMITTED` and above prevent it.

- Q: Why wouldn't you always just use `SERIALIZABLE` for maximum safety? → A: It significantly reduces concurrency (more blocking/overhead), which can hurt throughput and increase contention-related failures/timeouts under load — the standard guidance is choosing the lowest isolation level that's actually safe for the specific operation, not defaulting to the strictest.
- Q: What's the difference between pessimistic locking (`SELECT ... FOR UPDATE`) and optimistic locking (a version column)? → A: Pessimistic locking blocks other transactions upfront, preventing conflicts from happening at all; optimistic locking lets transactions proceed independently and only detects a conflict at commit time (via a version/timestamp check), retrying if one occurred — better concurrency when conflicts are rare, worse (wasted work) when they're common.

Tricky Questions

- Q: Does `REPEATABLE_READ` fully prevent phantom reads in every database? → A: Not universally by the strict SQL standard definition — some databases (notably MySQL's InnoDB) implement `REPEATABLE_READ` in a way that also prevents phantom reads via gap locking, going beyond what the standard technically requires at that level, while other databases follow the standard's table more literally; this is a genuine, documented cross-database inconsistency worth knowing for real production work.
- Q: If Connection A holds a `SELECT ... FOR UPDATE` lock and never commits or rolls back, what happens to Connection B trying to update the same row? → A: It blocks indefinitely (or until its own configured timeout expires) — an un-committed, forgotten pessimistic lock is a real, serious production incident pattern (effectively a database-level deadlock/hang risk), which is exactly why transactions must always be reliably committed or rolled back, ideally with a reasonable timeout configured.

Coding Exercise

L1: Print your database's default isolation level and set it explicitly to `READ_COMMITTED` and `SERIALIZABLE`, observing the returned constants. **L2:** Research (via documentation) your specific database's approach to isolation (locking vs MVCC) and summarize it in your own words. **L3:** Write the SQL for a pessimistic-locking checkout flow (`SELECT ... FOR UPDATE` then conditional `UPDATE`) for a stock-decrement scenario. **L4 (Interview):** Explain dirty read, non-repeatable read, and phantom read with a concrete example of each. **L5 (Senior):** Design a checkout system's concurrency strategy, choosing between pessimistic locking and optimistic locking (version column) for stock decrement, and justify your choice given expected contention levels. **L6 (Mastery):** Recreate the isolation levels table from memory, including which anomalies each level prevents.

CHAPTER 7 — Connection Pooling

Telugu Explanation

Ch.2 లో చూసినట్లు, Connection create చేయడం expensive. **Connection Pool** ఒక fixed number of connections ని **pre-create చేసి, reuse చేస్తుంది** — application connection "borrow" చేస్తుంది, పని అయ్యాక "return" చేస్తుంది (అసలు close చేయదు, pool కి తిరిగి ఇస్తుంది). ఇది Book 08, Ch.1 లో చూసిన thread pool concept కి direct analog — expensive resources reuse చేయడం.

Professional English Explanation

As established in Ch.2, creating a Connection is expensive. A **Connection Pool** pre-creates and reuses a fixed number of connections — the application "borrows" one, uses it, and "returns" it (rather than actually closing it — the pool keeps it open for the next borrower). This is a direct analog to Book 08, Ch.1's thread pool concept — reusing an expensive resource instead of paying its creation cost repeatedly.

Java Code — Using HikariCP (the modern standard connection pool)

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.*;

public class ConnectionPoolingDemo {
    public static void main(String[] args) throws SQLException {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:h2:mem:testdb6;DB_CLOSE_DELAY=-1");
        config.setUsername("sa");
        config.setPassword("");
        config.setMaximumPoolSize(10); // like ThreadPoolExecutor's maxPoolSize
        config.setMinimumIdle(2); // connections kept ready even when
        config.setConnectionTimeout(30_000); // max wait time to borrow a
        config.setIdleTimeout(600_000); // how long an idle connection may sit
        config.setMaxLifetime(1_800_000); // max total lifetime of a connection
        try (HikariDataSource dataSource = new HikariDataSource(config)) {
            try (Connection conn = dataSource.getConnection()) { // "borrow"
                try (Statement stmt = conn.createStatement()) {
                    stmt.execute("CREATE TABLE ping (id INT)");
                    stmt.execute("INSERT INTO ping VALUES (1)");
                }
            } // "return"
            System.out.println("Active connections: " +
                dataSource.getHikariPoolMXBean().getActiveConnections());
            System.out.println("Idle connections: " +
                dataSource.getHikariPoolMXBean().getIdleConnections());
            System.out.println("Total connections: " +
                dataSource.getHikariPoolMXBean().getTotalConnections());

            // Simulating multiple "requests" reusing pooled connections
            for (int i = 0; i < 5; i++) {
                try (Connection conn = dataSource.getConnection()) {
                    System.out.println("Request " + i + " borrowed a connection (likely reused,
                        not newly created)");
                }
            }
        }
    }
}
```

Output (illustrative — requires the HikariCP dependency on the classpath)

```
Active connections: 0
Idle connections: 1
Total connections: 1
Request 0 borrowed a connection (likely reused, not newly created)
Request 1 borrowed a connection (likely reused, not newly created)
Request 2 borrowed a connection (likely reused, not newly created)
Request 3 borrowed a connection (likely reused, not newly created)
Request 4 borrowed a connection (likely reused, not newly created)
Request 4 borrowed a connection (likely reused, not newly created)
```

Internal Working

- Calling `.close()` on a **pooled** connection (obtained from `dataSource.getConnection()`) does **not** actually close the underlying database connection — HikariCP wraps the real connection in a proxy that intercepts `.close()` and instead **returns it to the pool** for reuse, resetting its state (auto-commit, transaction isolation, etc.) to defaults first — this is precisely why try-with-resources remains the correct idiom even with pooling, but the actual effect underneath is fundamentally different from a raw `DriverManager` connection's `.close()`.
- `maximumPoolSize` must be tuned deliberately — too small, and requests queue/timeout waiting for a connection under load; too large, and you risk exhausting the *database server's* own max-connections limit (which is often shared across multiple application instances) — this mirrors Book 08, Ch.1's `ThreadPoolExecutor` sizing trade-offs exactly, and in fact a common production guideline is that pool size should relate to the number of CPU cores and typical query latency, not simply "as large as possible."
- `maxLifetime` exists to proactively recycle connections before they go stale (e.g., a load balancer or firewall silently dropping long-idle connections, or a database server's own connection-lifetime limits) — a subtle, real production reliability concern that a naive "keep connections forever" pool implementation would miss.

Real-World Example

Telugu: Spring Boot applications (Book 12) default నా HikariCP వాడతాయి — `DataSource` bean అంతర్గత నా HikariCP pool. Production tuning (pool size, timeout values) app యొక్క actual traffic pattern, DB server capacity మీద ఆధారపడి చేయాలి, default values మీద గుడ్డిగా ఆధారపడకూడదు. English: Spring Boot applications (Book 12) use HikariCP as the default connection pool implementation behind the `DataSource` bean — production tuning (pool size, timeouts) should be based on the application's actual traffic pattern and the database server's real capacity, never left blindly on defaults, exactly the same operational discipline as Book 08, Ch.1's thread pool tuning.

Interview Answer

"A connection pool pre-creates and reuses a fixed number of database connections instead of paying the expensive connection-establishment cost per operation. Calling `.close()` on a pooled connection doesn't actually close it — it returns it to the pool after resetting its state. Pool size must be deliberately tuned, balancing application throughput needs against the database server's own

connection limits — a direct analog to Book 08's thread pool sizing trade-offs. HikariCP is the modern standard implementation, used by default in Spring Boot."

Cross Questions

- Q: Does calling `.close()` on a connection from a pool actually terminate the database connection? → A: No — it returns the connection to the pool for reuse (after resetting its state to defaults); the underlying database connection stays open until the pool itself decides to recycle/close it (e.g., due to `maxLifetime` or pool shutdown).
- Q: Why can't you just set `maximumPoolSize` extremely high "to be safe"? → A: The database server has its own finite max-connections limit, often shared across multiple application instances — an oversized pool (especially multiplied across several app instances) risks exhausting that shared limit, causing failures across the whole system, not just your one application.
- Q: What problem does `maxLifetime` solve? → A: It proactively recycles connections before they become stale due to network infrastructure (load balancers, firewalls) or database-side connection-age limits silently dropping very long-lived connections, which would otherwise cause confusing "connection is closed" errors on next use.

Tricky Questions

- Q: If a pooled connection's auto-commit was set to `false` and never explicitly reset before being "closed" (returned to the pool), what happens to the next borrower? → A: A well-implemented pool (like HikariCP) automatically resets connection state (auto-commit, isolation level, etc.) to configured defaults when a connection is returned — but relying on this without verifying your specific pool's behavior is risky; explicitly resetting critical state yourself before returning a connection is the safer practice.
- Q: Can connection pool exhaustion (`connectionTimeout` expiring while waiting to borrow) cause a cascading failure across a microservices system? → A: Yes — this is a real, documented production failure pattern: a slow downstream dependency causes connections to be held longer than usual, exhausting the pool, causing timeouts and retries that further increase load, a feedback loop connecting directly to Book 16's resilience patterns (circuit breakers, timeouts).

Coding Exercise

L1: Set up a HikariCP-backed connection pool and observe active/idle/total connection counts via `HikariPoolMXBean`. **L2:** Deliberately set `maximumPoolSize` very small (e.g., 1) and simulate multiple concurrent borrowers to observe queuing/timeout behavior. **L3:** Research and document your understanding of how `maxLifetime` and `idleTimeout` interact to keep a pool healthy over a long-running application's lifetime. **L4 (Interview):** Explain why closing a pooled connection doesn't actually close the underlying database connection. **L5 (Senior):** Design a connection pool sizing strategy for a service expecting 200 concurrent requests with typical query latency of 20ms, justifying your chosen pool size. **L6 (Mastery):** Explain, from memory, the connection-pool-exhaustion cascading-failure pattern and its connection to Book 16's resilience patterns.

CHAPTER 8 — SQL Fundamentals Refresher: Joins & Normalization

Telugu Explanation

Relational databases **normalized** tables గా design అవుతాయి — data duplication తగ్గించడానికి, related data వేర్వేరు tables లో store అయ్యి, **foreign keys** ద్వారా link అవుతాయి. **Joins** ఈ వేర్వేరు tables నుండి data ని కలిపి query చేయడానికి వాడతారు — **INNER JOIN** (matching rows మాత్రమే), **LEFT JOIN** (left table యొక్క అన్ని rows, right side match లేకపోయినా NULL తో), **RIGHT JOIN**, **FULL OUTER JOIN**.

Professional English Explanation

Relational databases are designed with **normalized** tables — reducing data duplication by splitting related data into separate tables, linked via **foreign keys**. **Joins** combine data from these separate tables in a single query — **INNER JOIN** (only matching rows), **LEFT JOIN** (all rows from the left table, **NULL**-filled where there's no match on the right), **RIGHT JOIN**, **FULL OUTER JOIN**.

SQL Code

```
-- Normalized schema: Customer and Order are separate tables, linked by a foreign key
CREATE TABLE customer (id INT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));
CREATE TABLE orders (id INT PRIMARY KEY, customer_id INT, total DOUBLE, FOREIGN KEY
(customer_id) REFERENCES customer(id));

INSERT INTO customer VALUES (1, 'Ravi', 'ravi@example.com'), (2, 'Anitha',
'anitha@example.com'), (3, 'Kiran', 'kiran@example.com');
INSERT INTO orders VALUES (101, 1, 500.0), (102, 1, 1200.0), (103, 2, 300.0);
-- Note: Kiran (id=3) has NO orders yet - useful for demonstrating LEFT JOIN below

-- INNER JOIN: only customers who HAVE placed orders
SELECT c.name, o.id AS order_id, o.total
FROM customer c
INNER JOIN orders o ON c.id = o.customer_id;
-- Result: Ravi/101/500, Ravi/102/1200, Anitha/103/300 (Kiran excluded - no matching order)

-- LEFT JOIN: ALL customers, with NULL order info if they have none
SELECT c.name, o.id AS order_id, o.total
FROM customer c
LEFT JOIN orders o ON c.id = o.customer_id;
-- Result: Ravi/101/500, Ravi/102/1200, Anitha/103/300, Kiran/NULL/NULL (Kiran included, order
fields NULL)

-- Aggregation with JOIN: total spent per customer, including customers with $0 (LEFT JOIN +
COALESCE)
SELECT c.name, COALESCE(SUM(o.total), 0) AS total_spent
FROM customer c
LEFT JOIN orders o ON c.id = o.customer_id
GROUP BY c.name;
-- Result: Ravi/1700, Anitha/300, Kiran/0
```

Java Code — Consuming a JOIN via JDBC

```
import java.sql.*;

public class JoinsNormalizationDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb7", "sa", "")) {
            try (Statement setup = conn.createStatement()) {
                setup.execute("CREATE TABLE customer (id INT PRIMARY KEY, name VARCHAR(100))");
                setup.execute("CREATE TABLE orders (id INT PRIMARY KEY, customer_id INT, total");
DOUBLE)");
                setup.execute("INSERT INTO customer VALUES (1, 'Ravi'), (2, 'Anitha'), (3,");
                setup.execute("INSERT INTO orders VALUES (101, 1, 500.0), (102, 1, 1200.0),");
                setup.execute("INSERT INTO orders VALUES (103, 2, 300.0)");
            }

            String sql = """
                SELECT c.name, COALESCE(SUM(o.total), 0) AS total_spent
                FROM customer c
                LEFT JOIN orders o ON c.id = o.customer_id
                GROUP BY c.name
                ORDER BY total_spent DESC
            """;
            // text
            block (Book 07, Ch.14)

            try (PreparedStatement pstmt = conn.prepareStatement(sql);
                ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getString("name") + ": " +
rs.getDouble("total_spent"));
                }
            }
        }
    }
}
```

Output

```
Ravi: 1700.0
Anitha: 300.0
Kiran: 0.0
```

Internal Working — Why Normalization Matters

- **Without normalization** (e.g., storing customer name/email directly on every order row), updating a customer's email would require updating **every single order row** for that customer — a maintenance nightmare and a data-integrity risk (some rows updated, others missed, leaving inconsistent duplicate data).
- **With normalization**, customer data lives in exactly **one place** (the `customer` table); orders merely *reference* it via `customer_id` — updating the email once automatically reflects correctly for every order, since queries `JOIN` to fetch it live, not from a stale copy.
- `INNER JOIN` vs `LEFT JOIN` is a frequent, meaningful correctness decision, not just syntax — using `INNER JOIN` when you actually need "all customers, including those with zero orders"

silently drops the zero-order customers from the result, a real, common reporting bug.

Real-World Example

Telugu: E-commerce schema — Customer , Order , Product , OrderItem — అన్నీ normalized, foreign keys తో linked. "Total spent per customer" report LEFT JOIN + GROUP BY + COALESCE వాడి, zero-order customers కూడా (₹0 తో) సరిగ్గా include చేయాలి — ఇదే production reporting లో అత్యంత common correctness bug source. English: A real e-commerce schema (Customer , Order , Product , OrderItem) is normalized with foreign-key relationships throughout; a "total spent per customer" report needs LEFT JOIN + GROUP BY + COALESCE to correctly include zero-order customers at ₹0 rather than silently dropping them — exactly the kind of subtle INNER vs LEFT JOIN mistake that's a genuinely common real reporting bug in production systems.

Interview Answer

"Normalization splits related data into separate tables linked by foreign keys, avoiding duplication and the update-anomaly risk of storing the same data redundantly across many rows. Joins recombine that data for querying — INNER JOIN returns only matching rows across both tables, LEFT JOIN returns all rows from the left table with NULLs where there's no match on the right, which matters whenever a report needs to include entities with zero related records."

Cross Questions

- Q: What problem does normalization solve? → A: Update anomalies and data duplication — storing the same fact (like a customer's email) redundantly across many rows risks inconsistency when only some copies get updated.
- Q: When would using INNER JOIN instead of LEFT JOIN produce an incorrect report? → A: Whenever the report needs to include entities with zero related records (e.g., customers with no orders) — INNER JOIN silently excludes them, while LEFT JOIN correctly includes them with NULL/zero values for the missing side.
- Q: Why use COALESCE(SUM(o.total), 0) instead of just SUM(o.total)? → A: SUM() over zero matching rows (from a LEFT JOIN producing NULLs) returns NULL, not 0 — COALESCE converts that NULL into a meaningful 0 for customers with no orders.

Tricky Questions

- Q: Does over-normalization ever cause its own problems? → A: Yes — extremely fine-grained normalization can require many joins for even simple queries, hurting both query complexity and performance; real-world schema design often deliberately denormalizes specific, well-understood hot paths for performance (a genuine trade-off, revisited in Book 21's system design context).
- Q: In a LEFT JOIN, if the right table has multiple matching rows for one left-table row, what happens? → A: The left-table row is duplicated once per matching right-table row in the result set (a classic "join fan-out"), which is exactly why aggregation (GROUP BY) is often needed afterward to collapse back to one row per left-table entity, as shown in the total-spent example.

Coding Exercise

L1: Create a normalized 2-table schema (Customer/Order) and write both `INNER JOIN` and `LEFT JOIN` queries, comparing their results. **L2:** Write a "total spent per customer" report using `LEFT JOIN + GROUP BY + COALESCE`, verifying zero-order customers show correctly. **L3:** Consume the JOIN query results via JDBC, mapping to a `CustomerSpendingSummary` record. **L4 (Interview):** Explain the difference between `INNER JOIN` and `LEFT JOIN` with a concrete example where the choice changes the result. **L5 (Senior):** Design a normalized schema for a small library system (Book, Author, Member, Loan), identifying foreign key relationships. **L6 (Mastery):** Explain, from memory, the update-anomaly problem normalization solves, and the "join fan-out" phenomenon with aggregation.

CHAPTER 9 — Indexes & Query Optimization

Telugu Explanation

Index అంటే ఒక table మీద ఒక (లేదా ఎక్కువ) columns మీద create చేసే **auxiliary data structure** (typically B-Tree) — full table scan చేయకుండా, target rows ని quickly locate చేయడానికి. Index లేకపోతే, `WHERE` clause query ప్రతి row ని check చేయాలి (**$O(n)$ full table scan**); index ఉంటే, **$O(\log n)$** లో target rows కనుక్కోగలదు.

Professional English Explanation

An index is an auxiliary data structure (typically a **B-Tree**) built on one or more columns, letting the database locate target rows quickly without scanning the entire table. Without an index, a `WHERE` clause query must check every row (**$O(n)$ full table scan**); with an appropriate index, it can locate matching rows in **$O(\log n)$** .

SQL Code

```
-- Without an index: every query on 'email' scans the ENTIRE table
CREATE TABLE customer (id INT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));
-- SELECT * FROM customer WHERE email = 'ravi@example.com'; --  $O(n)$  full table scan without
an index

-- Creating an index speeds up lookups on that column dramatically
CREATE INDEX idx_customer_email ON customer(email);
-- SELECT * FROM customer WHERE email = 'ravi@example.com'; -- now  $O(\log n)$  via the B-Tree
index

-- Composite (multi-column) index - column ORDER matters!
CREATE INDEX idx_order_customer_date ON orders(customer_id, order_date);
-- This index EFFICIENTLY serves:
-- WHERE customer_id = ? (uses the index - leading column)
-- WHERE customer_id = ? AND order_date = ? (uses the index fully - both columns)
-- This index CANNOT efficiently serve:
-- WHERE order_date = ? (order_date is NOT the leading column - index
mostly unused)

-- EXPLAIN reveals whether a query actually uses an index or falls back to a full scan
EXPLAIN SELECT * FROM customer WHERE email = 'ravi@example.com';
```

Java Code — Observing the Practical Impact

```
import java.sql.*;

public class IndexesQueryOptimizationDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection conn = DriverManager.getConnection("jdbc:h2:mem:testdb8", "sa", "")) {
            try (Statement setup = conn.createStatement()) {
                setup.execute("CREATE TABLE customer (id INT PRIMARY KEY, email VARCHAR(100))");
                for (int i = 1; i <= 50_000; i++) {
                    setup.execute("INSERT INTO customer VALUES (" + i + ", 'user" + i + "@example.com')");
                }
            }

            long start = System.nanoTime();
            queryByEmail(conn, "user49999@example.com");
            long noIndexTime = System.nanoTime() - start;
            System.out.println("Query time WITHOUT index: " + noIndexTime / 1_000_000 + " ms");

            try (Statement stmt = conn.createStatement()) {
                stmt.execute("CREATE INDEX idx_email ON customer(email)");
            }

            start = System.nanoTime();
            queryByEmail(conn, "user49999@example.com");
            long withIndexTime = System.nanoTime() - start;
            System.out.println("Query time WITH index: " + withIndexTime / 1_000_000 + " ms");
        }

        static void queryByEmail(Connection conn, String email) throws SQLException {
            try (PreparedStatement pstmt = conn.prepareStatement("SELECT id FROM customer WHERE email = ?")) {
                pstmt.setString(1, email);
                try (ResultSet rs = pstmt.executeQuery()) { rs.next(); }
            }
        }
    }
}
```

Output (illustrative — actual numbers vary by machine/database)

```
Query time WITHOUT index: 42 ms
Query time WITH index: 1 ms
```

Internal Working

- A B-Tree index organizes indexed values in a sorted, balanced tree structure — very similar in spirit to `TreeMap` / `TreeSet` 's Red-Black Tree (Book 05, Ch.5/7), giving $O(\log n)$ search instead of $O(n)$ linear scanning; the trade-off (also analogous to Book 05's collections) is that indexes cost extra storage and slow down **writes** slightly (every `INSERT` / `UPDATE` / `DELETE` must also update the index structure) — so indexing every column "just in case" is itself an anti-pattern, not a free win.

- **Composite index column order matters critically:** an index on `(customer_id, order_date)` can efficiently serve queries filtering on `customer_id` alone, or `customer_id AND order_date` together, but **cannot** efficiently serve a query filtering on `order_date` alone — the index is only useful as a left-to-right prefix match, exactly like a phone book sorted by (last name, first name) not helping you find someone by first name alone.
- **EXPLAIN** (syntax varies slightly per database) reveals the database's actual query execution plan — showing whether a query used an available index or fell back to a full table scan, which is the standard, essential diagnostic tool for real query performance investigation, not guesswork.

Real-World Example

Telugu: Production databases growth తో, index లేని columns మీద WHERE clause queries dramatically slow అవుతాయి — table 100 rows నుండి 10 million rows కి grow అయితే, full table scan performance linear గా (లేదా worse గా) degrade అవుతుంది. ఇది production performance incidents కి అత్యంత common root cause. English: As production databases grow, `WHERE` clause queries on unindexed columns degrade dramatically — a query fine at 100 rows becomes a serious production performance problem at 10 million rows, since a full table scan's cost grows linearly (or worse) with table size. This is, in practice, one of the single most common root causes of real production database performance incidents.

Interview Answer

"An index is typically a B-Tree structure built on one or more columns, turning an $O(n)$ full table scan into an $O(\log n)$ lookup for queries filtering on that column. Indexes cost extra storage and slow down writes slightly, so indexing every column isn't free and isn't automatically beneficial. Composite index column order matters — an index is only useful as a left-to-right prefix match on its columns. **EXPLAIN** reveals whether a query actually uses an available index."

Cross Questions

- Q: Why don't you just index every column in every table? → A: Every index adds storage overhead and slows down every write (INSERT/UPDATE/DELETE) to that table, since the index structure must also be updated — indexing should be deliberate, based on actual query patterns, not blanket applied.
- Q: For a composite index on `(a, b)`, can a query filtering only on `b` use it efficiently? → A: Generally no — composite indexes are only efficiently usable as a left-to-right prefix, so a query needing to filter by `b` alone would need a separate index on `b` (or `(b, a)`) to be efficient.
- Q: What does **EXPLAIN** show, and why is it essential for query optimization? → A: The database's actual execution plan — whether it used an index, what type of scan/join strategy it chose, and estimated costs — turning query optimization from guesswork into an evidence-based diagnostic process.

Tricky Questions

- Q: Does having an index on a column GUARANTEE the database will use it for every query on that column? → A: No — the query optimizer decides based on estimated cost; for very small

tables, or queries expected to match a large fraction of rows anyway, a full table scan can actually be cheaper than using the index (avoiding the overhead of index lookups plus subsequent row fetches), and the optimizer may correctly choose to ignore an available index in such cases.

- Q: Can too many indexes on a heavily-written table cause a real production problem? → A: Yes — a table with many indexes pays the write-overhead cost (updating every index) on every single INSERT/UPDATE/DELETE, which can become a genuine throughput bottleneck on write-heavy tables; index strategy must balance read-query needs against write-throughput needs, not optimize purely for reads.

Coding Exercise

L1: Reproduce the indexed-vs-unindexed query timing comparison from the demo on a larger dataset. **L2:** Create a composite index and write one query that benefits from it and one that doesn't (due to column order), verifying via `EXPLAIN`. **L3:** Use `EXPLAIN` on 3 different queries against an indexed table and interpret the output. **L4 (Interview):** Explain why indexing every column isn't a free performance win. **L5 (Senior):** Given a slow production query report, describe your diagnostic process using `EXPLAIN` to determine whether a missing/misused index is the root cause, and how you'd validate a proposed fix. **L6 (Mastery):** Explain, from memory, why composite index column order matters, using the phone-book analogy or an equivalent of your own.

CHAPTER 10 — Production JDBC Patterns + Mini Project

Telugu Explanation

Real production code raw JDBC ని ఇలా వాడదు — ప్రతి method లో connection/statement/resultset repeat చేయదు. బదులుగా, **DAO (Data Access Object) pattern** వాడతారు — database access logic ని ఒక్క, focused layer లో encapsulate చేసి (Book 02, Ch.14's coupling/cohesion directly apply అవుతుంది), business logic నుండి వేరు చేస్తారు.

Professional English Explanation

Real production code doesn't repeat raw connection/statement/result-set handling in every method. Instead, the **DAO (Data Access Object) pattern** encapsulates database access logic in one focused layer (directly applying Book 02, Ch.14's coupling/cohesion principles), cleanly separated from business logic.

Java Code — DAO Pattern

```
import java.sql.*;
import java.util.*;
import javax.sql.DataSource;

record Employee(int id, String name, double salary) {}

interface EmployeeDao {                                     // contract - generic
    Repository pattern (Book 06, Ch.6)
    void save(Employee employee) throws SQLException;
    Optional<Employee> findById(int id) throws SQLException;    // Optional (Book 07, Ch.9) -
                                                                // might not exist
    List<Employee> findAll() throws SQLException;
    void updateSalary(int id, double newSalary) throws SQLException;
}

class JdbcEmployeeDao implements EmployeeDao {
    private final DataSource dataSource;                     // injected connection
    pool (Ch.7), not created here

    JdbcEmployeeDao(DataSource dataSource) { this.dataSource = dataSource; }

    @Override
    public void save(Employee employee) throws SQLException {
        String sql = "INSERT INTO employee (id, name, salary) VALUES (?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, employee.id());
            pstmt.setString(2, employee.name());
            pstmt.setDouble(3, employee.salary());
            pstmt.executeUpdate();
        }
    }

    @Override
    public Optional<Employee> findById(int id) throws SQLException {
        String sql = "SELECT id, name, salary FROM employee WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, id);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(mapRow(rs));
                }
                return Optional.empty();
            }
        }
    }

    @Override
    public List<Employee> findAll() throws SQLException {
        String sql = "SELECT id, name, salary FROM employee ORDER BY id";
        List<Employee> result = new ArrayList<>();
        try (Connection conn = dataSource.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) result.add(mapRow(rs));
        }
        return List.copyOf(result);                          // defensive immutable
    }
}
```

```

return (Book 05, Ch.12)
}

@Override
public void updateSalary(int id, double newSalary) throws SQLException {
    String sql = "UPDATE employee SET salary = ? WHERE id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setDouble(1, newSalary);
        pstmt.setInt(2, id);
        int rowsAffected = pstmt.executeUpdate();
        if (rowsAffected == 0) throw new NoSuchElementException("No employee with id " +
id);
    }
}

private Employee mapRow(ResultSet rs) throws SQLException {           // ONE place mapping
ResultSet -> domain object
    return new Employee(rs.getInt("id"), rs.getString("name"), rs.getDouble("salary"));
}
}

public class ProductionJdbcPatternsDemo {
    public static void main(String[] args) throws SQLException {
        com.zaxxer.hikari.HikariConfig config = new com.zaxxer.hikari.HikariConfig();
        config.setJdbcUrl("jdbc:h2:mem:testdb9;DB_CLOSE_DELAY=-1");
        config.setUsername("sa");
        config.setPassword("");

        try (com.zaxxer.hikari.HikariDataSource dataSource = new
com.zaxxer.hikari.HikariDataSource(config)) {
            try (Connection conn = dataSource.getConnection(); Statement stmt =
conn.createStatement()) {
                stmt.execute("CREATE TABLE employee (id INT PRIMARY KEY, name VARCHAR(100),
salary DOUBLE)");
            }

            EmployeeDao dao = new JdbcEmployeeDao(dataSource);           // business logic
depends on the INTERFACE
            dao.save(new Employee(1, "Ravi", 70000.0));
            dao.save(new Employee(2, "Anitha", 85000.0));

            System.out.println("All: " + dao.findAll());
            System.out.println("Found by id: " + dao.findById(1));
            System.out.println("Not found: " + dao.findById(999));

            dao.updateSalary(1, 75000.0);
            System.out.println("After update: " + dao.findById(1));
        }
    }
}

```

Output

```

All: [Employee[id=1, name=Ravi, salary=70000.0], Employee[id=2, name=Anitha, salary=85000.0]]
Found by id: Optional[Employee[id=1, name=Ravi, salary=70000.0]]
Not found: Optional.empty
After update: Optional[Employee[id=1, name=75000.0]]

```


Internal Working & Design Rationale

- The `EmployeeDao` **interface** (Book 02, Ch.10–11) lets business/service-layer code depend on an abstraction, not the concrete `JdbcEmployeeDao` implementation — directly applying Book 02, Ch.13's Dependency Inversion Principle; this is precisely what makes it trivial to substitute a mock implementation for unit testing (Book 15) or swap to a JPA-backed implementation (Book 13) later, without touching any calling code.
- Every DAO method **fully manages its own resources** (opening and closing `Connection` / `Statement` / `ResultSet` within the method, via try-with-resources) rather than leaking connection-management concerns into callers — this is the correct resource-ownership boundary for this pattern.
- The private `mapRow()` helper centralizes `ResultSet`-to-domain-object mapping in exactly **one place** per DAO, avoiding the duplication (and duplicated-bug risk) of repeating that mapping logic across `findById()` and `findAll()` separately.
- Using `Optional<Employee>` for `findById()` (Book 07, Ch.9) rather than returning `null` correctly signals "might not exist" at the type level — exactly the idiom that chapter established as the correct use of `Optional`.

Mini Project: Banking Ledger CRUD Application

Goal: Combine every concept in this book into one realistic JDBC-based application.

Requirements:

1. Schema: `account(id, owner_name, balance)` and `transaction_log(id, account_id, type, amount, timestamp)` — normalized (Ch.8), with a foreign key.
2. `AccountDao` interface + `JdbcAccountDao` implementation (Ch.10's DAO pattern) with `save`, `findById` (returning `Optional`), `findAll`, `updateBalance`.
3. A `TransferService` class (business logic layer, separate from the DAO) implementing `transferMoney(fromId, toId, amount)`:
 - Uses a single `Connection` with manual transaction control (Ch.5) — both the debit and credit, plus two `transaction_log` inserts, commit or roll back together.
 - Uses `SELECT ... FOR UPDATE` (Ch.6) to pessimistically lock both accounts (in a **consistent order by ID**, directly reusing Book 08 Part 1, Ch.9's deadlock-avoidance principle) before reading/updating balances.
 - Validates sufficient balance, throwing a custom `InsufficientFundsException` (Book 04, Ch.6) that triggers rollback.
4. All queries use `PreparedStatement` exclusively (Ch.3) — zero string concatenation of any dynamic value into SQL.
5. Uses a HikariCP connection pool (Ch.7), properly configured and shut down.
6. A report method using a `LEFT JOIN` (Ch.8) to show every account's current balance alongside its total historical transaction count (including accounts with zero transactions).
7. An index (Ch.9) on `transaction_log(account_id)`, justified by the report query's access pattern.

Concepts Reinforced: Every chapter in this book, applied together in one realistic, correctly-transactional financial application.

Stretch Goal: Add optimistic locking (a `version` column on `account`) as an alternative to the pessimistic `SELECT ... FOR UPDATE` approach, and discuss (in comments) the trade-off between the two for this specific use case.

FINAL REVISION NOTES

- **JDBC architecture:** `java.sql` interfaces (database-agnostic) + vendor `Driver` (database-specific); auto-registered since JDBC 4.0.
 - **Connection:** expensive to create; auto-commit is on by default (each statement its own implicit transaction).
 - **PreparedStatement:** structurally prevents SQL Injection (data and SQL syntax sent separately) — mandatory for any query involving user input, not optional.
 - **ResultSet:** forward-only cursor by default, 1-indexed columns, `wasNull()` needed to disambiguate primitive-getter 0/false from actual SQL NULL.
 - **ACID:** Atomicity/Consistency/Isolation/Durability; `rollback()` undoes everything since the transaction began; always pair commit/rollback in try/catch.
 - **Isolation levels:** READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE, trading consistency for concurrency; dirty/non-repeatable/phantom reads prevented progressively.
 - **Connection pooling:** reuse expensive connections; `.close()` returns to pool, doesn't truly close; size deliberately, mirroring Book 08's thread-pool tuning.
 - **Joins/normalization:** normalize to avoid update anomalies; `INNER` vs `LEFT JOIN` is a correctness decision, not just syntax; `COALESCE` handles NULL from unmatched `LEFT JOIN` rows.
 - **Indexes:** B-Tree, $O(\log n)$ vs $O(n)$ full scan; composite index column order matters (left-to-right prefix only); not free — costs storage and write overhead; `EXPLAIN` reveals actual usage.
 - **DAO pattern:** interface + implementation, one place for `ResultSet` mapping, resources fully self-managed, Optional for "might not exist" — directly built on Book 02's DIP and Book 07's Optional idiom.
-



CHEAT SHEET

JDBC: java.sql interfaces (agnostic) + vendor Driver (specific), auto-registered since JDBC 4.0
Connection: expensive, auto-commit=true by default | Connection/Statement/ResultSet all AutoCloseable
PreparedStatement: STRUCTURAL SQL injection defense (data/SQL sent separately) - mandatory for user input
ResultSet: forward-only cursor default, 1-INDEXED columns, wasNull() for primitive-getter NULL ambiguity
ACID: Atomicity(all-or-nothing) Consistency(valid states) Isolation(no interference) Durability(survives crash)
rollback() undoes everything since transaction start | always:
try{...commit()}catch{rollback();throw}finally{autoCommit=true}
Isolation levels (weak->strong): READ_UNCOMMITTED < READ_COMMITTED < REPEATABLE_READ < SERIALIZABLE
prevents progressively: dirty read -> non-repeatable read -> phantom read
Connection pool: reuse expensive connections, .close()=return not destroy, size deliberately (like thread pools)
INNER JOIN=matching rows only | LEFT JOIN=all left rows + NULL fill | COALESCE(x,0) fixes NULL from unmatched LEFT JOIN
Normalization: avoid update anomalies, split data + foreign keys | over-normalization hurts query complexity
Index: B-Tree, $O(\log n)$ vs $O(n)$ scan | composite index = LEFT-TO-RIGHT PREFIX only | costs write overhead, not free
EXPLAIN shows actual query plan / index usage
DAO pattern: interface (DIP) + impl, ResultSet mapping in ONE place, Optional for "might not exist", self-managed resources



INTERVIEW QUESTION BANK — JDBC & Database

Beginner

1. What is JDBC, and what problem does it solve?
2. What is the difference between `Statement` and `PreparedStatement` ?
3. What does `ResultSet.next()` do, and what does it return?
4. What is a transaction, and what do commit/rollback do?
5. What is the difference between `INNER JOIN` and `LEFT JOIN` ?

Intermediate 6. Explain why `PreparedStatement` prevents SQL Injection at a structural level. 7. Explain all 4 ACID properties with an example of what breaks without each. 8. Why is closing a pooled connection different from closing a raw JDBC connection? 9. What problem does normalization solve, and what's the trade-off of over-normalizing? 10. Explain why composite index column order matters.

Advanced 11. Explain dirty read, non-repeatable read, and phantom read, and which isolation levels prevent each. 12. Explain the difference between pessimistic locking (`SELECT ... FOR UPDATE`) and optimistic locking (version column), and when you'd choose each. 13. Why does `wasNull()` need to be called after `ResultSet.getInt()` on a nullable column? 14. Explain connection pool sizing trade-offs and the risk of pool exhaustion cascading into a larger system failure. 15. Explain when a database's query optimizer might legitimately choose NOT to use an available index.

Senior/Architect 16. Design a deadlock-free money-transfer method using pessimistic locking and consistent lock ordering by account ID. 17. Diagnose a production report showing incorrect (too-low) customer counts — walk through how an `INNER JOIN` vs `LEFT JOIN` mistake could cause this. 18. Design a connection pool sizing and timeout strategy for a service with known traffic patterns and database capacity limits. 19. Review a DAO layer with raw JDBC scattered across service classes — propose a refactoring plan using the DAO pattern and dependency inversion. 20. Explain, end-to-end, how you would diagnose and fix a slow production query using `EXPLAIN`, connecting the fix to proper index design.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is a transaction? → Q: What does ACID stand for? → Q: What does rollback() actually undo? → Q: Why must auto-commit be disabled for multi-statement atomicity? → Q: What happens if you forget to call commit() or rollback()?

Q: Why use PreparedStatement over Statement? → Q: How exactly does it prevent SQL Injection? → Q: Does it protect dynamic table/column names too? → Q: What performance benefit does it offer beyond security? → Q: What is batch execution, and why is it faster?

Q: What is an index? → Q: What data structure does it typically use? → Q: Why isn't indexing every column a good idea? → Q: Why does composite index column order matter? → Q: How would you verify whether a query is actually using an index?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every SQL/JDBC decision out loud in Telugu. **L3 — Advanced:** Build a full DAO layer (interface + JDBC implementation) for a 2-table normalized schema. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 10 Banking Ledger mini project fully, including the optimistic-locking stretch goal. **L6 — Mastery:** Teach Chapters 5 (transactions/ACID), 6 (isolation levels), and 9 (indexes) out loud, from memory, using fresh examples.



ONE-DAY REVISION PLAN (≈5 hours)

Time	Focus
0:00–0:30	Ch.1–2: JDBC architecture, connecting — redraw the architecture diagram
0:30–1:00	Ch.3: Statement/PreparedStatement/CallableStatement — reproduce the SQL Injection demo
1:00–1:30	Ch.4: ResultSet — memorize the wasNull()/1-indexing gotchas
1:30–1:45	Break
1:45–2:30	Ch.5: Transactions/ACID — the highest-density interview block
2:30–3:00	Ch.6: Isolation levels — memorize the anomaly-prevention table
3:00–3:30	Ch.7: Connection pooling
3:30–4:00	Ch.8: Joins/normalization
4:00–4:30	Ch.9: Indexes & query optimization
4:30–5:00	Ch.10 + Interview Bank — DAO pattern review, answer questions from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–2 (architecture, connecting) — set up H2, run every example yourself
2	Ch.3–4 (statement types, ResultSet) — reproduce and fix the SQL Injection demo
3	Ch.5–6 (transactions/ACID, isolation levels) — implement the bank transfer with rollback
4	Ch.7 (connection pooling) — set up HikariCP, tune and observe pool metrics
5	Ch.8–9 (joins/normalization, indexes) — build a normalized schema and benchmark indexed vs unindexed queries
6	Ch.10 + Mini Project — build the full Banking Ledger CRUD application with the stretch goal
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes



FINAL MASTERY CHECKLIST

- ☐ I can draw the JDBC architecture diagram from memory.
- ☐ I can explain exactly why PreparedStatement prevents SQL Injection structurally.
- ☐ I can correctly handle ResultSet NULL ambiguity and 1-based indexing.
- ☐ I can explain all 4 ACID properties with a concrete failure example for each.
- ☐ I can explain dirty/non-repeatable/phantom reads and which isolation levels prevent each.
- ☐ I can explain why connection pooling matters and how pooled close() differs from raw close().
- ☐ I can write correct INNER/LEFT JOIN queries and explain the normalization trade-off.
- ☐ I can explain how indexes work, including composite index column order.
- ☐ I can design a proper DAO layer using interfaces, Optional, and self-managed resources.
- ☐ I built the Banking Ledger CRUD mini project, including the optimistic-locking stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for [10_Advanced_Java_Backend.md](#) .